Now We Are Presenting

Logical & Shift Instructions in ARM Cortex-M3

AND, ORR, EOR, BIC, ORN (3.5.2) + ASR, LSL, LSR, ROR, RRX (3.5.3)

TEAM 11 | Microcontroller Course | Ref: ST PM0056 Rev 7, pages 75-76

Rasel Ahammed - 2310776141

Md. Shoriful Islam - 2310376113

Mahamudul Hasan Masum - 2310876111

What We Will Cover

1. **Logical instructions**
   AND, ORR, EOR, BIC, ORN
2. **Shift & rotate**
   ASR, LSL, LSR, ROR, RRX
3. **Flags & restrictions**
   N, Z, C, V behaviour
4. **Worked examples**
   Binary + math
5. **Real use + summary**
   Bit set / clear / toggle

3.5.2 Logical Instructions

Bitwise operations: each bit of Rn is combined with the same bit of Operand2

```
op{S}{cond} {Rd,} Rn, Operand2
```

Syntax parts

- op: AND, ORR, EOR, BIC, ORN
- S: update flags (optional)
- cond: condition code, e.g. EQ (optional)
- Rd: destination register
- Rn: register with 1st operand
- Operand2: flexible 2nd operand (register, #imm, shifted reg)

Operation

- AND: bitwise AND
- ORR: bitwise OR (bit set)
- EOR: bitwise exclusive OR
- BIC: Rn AND NOT Operand2 (bit clear)
- ORN: Rn OR NOT Operand2

AND, ORR, EOR

Truth table + meaning

A	B	AND	ORR	EOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1

```
R2 = 0x1234ABCD
AND R9,R2,#0xFF00
 -> R9 = 0x0000AB00
(keeps bits 8-15 only)
```

Key uses

- AND: mask / keep only chosen bits
- ORR: set chosen bits to 1
- EOR: toggle chosen bits; same value gives 0
- Result goes to Rd; Rn and Operand2 unchanged

BIC and ORN

Both use the complement (NOT) of Operand2

BIC (Bit Clear)

- Rd = Rn AND (NOT Operand2)
- Where Operand2 bit = 1, result bit = 0

```
R1 = 0xFF
BIC R0,R1,#0xAB
NOT 0xAB = 0x54
0xFF AND 0x54
R0 = 0x54
```

ORN (OR NOT)

- Rd = Rn OR (NOT Operand2)
- Where Operand2 bit = 0, result bit = 1

```
R11 = 0x0000000F
ORN R7,R11,#0xF0
NOT 0xF0 = 0xFFFFFF0F
0x0F OR 0xFFFFFF0F
R7 = 0xFFFFFF0F
```

Logical Ops: Flags & Examples

If S is specified

- N and Z updated from result
- C may update while computing Operand2
- V flag is not affected

Restriction

- Do not use SP or PC as registers
- Condition suffix e.g. EQ runs only if Z = 1

```
AND   R9, R2, #0xFF00
ORREQ R2, R0, R5
ANDS  R9, R8, #0x19
EORS  R7, R11, #0x18181818
BIC   R0, R1, #0xab
ORN   R7, R11, R14, ROR #4
ORNS  R7, R11, R14, ASR #32
```

Worked Example: Logical Math

Binary view, 8 bits for clarity

Instruction	Rn	Operand2	Result
ANDS R9,R8,#0x19	0010 1101	0001 1001	0000 1001
ORR R0,R1,R2	1100 0011	0000 1111	1100 1111
EORS R7,R11,#0xFF	1010 1010	1111 1111	0101 0101

Check

- ANDS: result 0x09 is non-zero so Z = 0, N = 0
- EOR with all 1s = NOT (flips every bit)
- BIC with 0xAB clears bits 0,1,3,5,7

3.5.3 Shift & Rotate Instructions

Arithmetic shift right, logical shift left/right, rotate right, rotate right with extend

```
op{S}{cond} Rd, Rm, Rs
op{S}{cond} Rd, Rm, #n
RRX{S}{cond} Rd, Rm
```

Shift length n

- ASR, LSR: 1 to 32
- LSL: 0 to 31 ROR: 1 to 31

Operands

- Rd: destination register
- Rm: value to be shifted (unchanged)
- Rs: shift length register, only lowest byte used (0-255)
- n: constant shift length

Notes

- Result is written to Rd
- RRX shifts right by exactly 1
- MOV Rd, Rm is the preferred form of LSL Rd, Rm, #0

LSL and LSR

Logical shifts: empty bits are filled with 0

LSL (Shift Left)

- Bits move left, 0 enters on right
- Shift by n = multiply by 2^n

```
R2 = 0000 1011  (11)
LSLS R1,R2,#3
R1 = 0101 1000  (88)
11 x 2^3 = 88
C = last bit out
```

LSR (Shift Right)

- Bits move right, 0 enters on left
- Shift by n = unsigned divide by 2^n

```
R5 = 0x140  (320)
LSR R4,R5,#6
R4 = 5
320 / 2^6 = 5
```

ASR（Arithmetic Shift Right）

Sign bit is copied in, so signed numbers keep their sign

Explanation

- Bits move right by n
- Top bit (sign) is replicated
- Signed divide by 2^n (rounds toward minus infinity)
- Positive: same as LSR
- Negative stays negative

```
R8 = 0xFFFFF000
  = -4096
ASR R7,R8,#9
R7 = 0xFFFFFFF8
  = -8

-4096 / 2^9 = -8

LSR would give
0x007FFFF8 (wrong
for signed)
```

ROR and RRX

Rotate: bits that leave one side re-enter the other

ROR (Rotate Right)

- Bits shifted out of bit 0 enter bit 31
- No bit is lost, n = 1 to 31 (or Rs)

```
R5 = 0x00000001
R6 = 1
ROR R4,R5,R6
R4 = 0x80000000
```

RRX (Rotate Right Extend)

- Shift right by 1 through the C flag
- Old C goes to bit 31, bit 0 goes to C

```
R5 = 0x00000002, C = 1
RRX R4,R5
R4 = 0x80000001
(C was 1 -> bit 31)
```

Shift Instructions: Flags & Restrictions

If S is specified

- N and Z updated from result
- C = last bit shifted out
- Exception: shift length 0, C unchanged

Restriction

- Do not use SP or PC

```
ASR  R7, R8, #9
LSLS R1, R2, #3
LSR  R4, R5, #6
ROR  R4, R5, R6
RRX  R4, R5

; ROR by bottom byte of R6
; LSLS also updates flags
```

Quick Comparison

Instr	Effect	Math view
AND	keep / mask bits	A & B
ORR	set bits	A | B
EOR	toggle bits	A ^ B
BIC	clear bits	A & ~B
ORN	OR with NOT	A | ~B
LSL / LSR	shift, fill 0	x2^n / unsigned /2^n
ASR	shift, keep sign	signed /2^n
ROR / RRX	rotate	no bits lost

Real Use: Controlling Bits (e.g. GPIO)

Typical register bit manipulation in embedded code

```
; R0 = port register value
ORR R0,R0,#0x20  ; set bit 5
BIC R0,R0,#0x20  ; clear bit 5
EOR R0,R0,#0x20  ; toggle bit 5
ANDS R1,R0,#0x20 ; test bit 5
  BEQ off        ; Z=1 -> bit is 0

LSL R1,R2,#2     ; R2 x 4
LSR R3,R2,#1     ; R2 / 2
```

Why it matters

- Control pins, flags, timers
- Pack / unpack data fields
- Fast multiply/divide by powers of 2
- Needs no extra memory

Summary

- Logical ops work bit by bit: AND, ORR, EOR, BIC, ORN
- Shifts move bits: LSL, LSR, ASR; rotates: ROR, RRX
- S suffix updates N, Z (and C for shifts)
- Never use SP or PC with these instructions

Thank You

Team 11